Documentazione del Progetto: Basi di Dati

TAMAGOTCHI

Sistema di Gestione per Videogioco

Zazzaro Flavia (DE1000158)
Università degli Studi di Napoli Federico II

Zinno Raquel (DE1000012)
Università degli Studi di Napoli Federico II

Indice

1 Specifica dei Requisiti **4**
1.1 Scopo e ambito del sistema 4
1.2 Attori primari 4
1.3 Requisiti funzionali 4
1.4 Requisiti non funzionali 5

2 Diagramma dei casi d'uso **6**
2.1 Diagramma 6
2.2 Descrizione dei casi d'uso 6

3 Lista delle funzioni ad alto livello **11**
3.1 Gestione Utente 11
3.2 Gestione Animale 12
3.3 Gestione Cibo 13
3.4 Gestione Vestito 14
3.5 Gestione Minigame 15

4 Schema concettuale (Modello E-R iniziale) **16**

5 Regole di business **17**
5.1 Vincoli di base 17
5.2 Vincoli semantici 17

6 Schema concettuale ristrutturato **18**
6.1 Diagramma ristrutturato 18
6.2 Ristrutturazione 18

7 Schema logico del database **19**

8 Specifiche dettagliate delle operazioni critiche **21**
8.1 Operazione 1: creaAnimale 21
8.2 Operazione 2: compra 22
8.3 Operazione 3: usaItem 23

9 Implementazione dei vincoli sui dati **24**
9.1 Piano di Implementazione dei Vincoli 24
9.2 Script dei Trigger in PostgreSQL (PL/pgSQL) 25

10 Script SQL per la creazione dello Schema **28**

11 Script DML e Popolamento Dati **30**

1. SPECIFICA DEI REQUISITI

1.1 Scopo e ambito del sistema

Il sistema permette a un utente di creare uno o più animali virtuali (fino a un massimo di 2 per ogni utente) scegliendo fra due diverse tipologie, un orso e un pinguino. L'utente avrà uno username e una password associati e sarà capace, dopo l'accesso iniziale, di gestire la creazione e l'eliminazione degli animali, scegliere l'animale con cui giocare fra quelli creati, comprare degli item, dare da mangiare e vestire l'animale.

Ogni animale ha un nome scelto e modificabile dall'utente, un livello di energia e un livello di fame gestiti come interi. Inoltre, ognuno dei due tipi di animale gestisce i propri livelli in maniera differente. Col passare del tempo i livelli di fame ed energia si abbasseranno. Per riguadagnare l'energia l'animale può dormire, per poi svegliarsi successivamente. Invece per riguadagnare la fame l'utente può dargli da mangiare del cibo.

Tramite il sistema sarà possibile giocare a dei minigame, ognuno dotato di nome, ricompensa in punti ed energia consumata: in caso di vittoria, l'animale perderà energia ma guadagnerà dei punti; in caso di sconfitta perderà solamente energia.

I punti possono essere utilizzati dall'utente per comprare gli item. Questi hanno un costo e un nome, in più sono suddivisi in vestiti e cibo: del cibo bisogna conoscere il valore di punti fame che ricarica quando utilizzato e il tipo (salato, dolce o bevanda); invece i vestiti possono essere indossati dall'animale e possiedono dei valori di boost, poiché quando usati aumenteranno il valore massimo dei livelli di fame/energia.

1.2 Attori primari

- Utente anonimo
- Utente riconosciuto

1.3 Requisiti funzionali

- Crea account
- Accede all'account
- Cambia password
- Crea l'animale
- Esce dall'account
- Seleziona l'animale
- Elimina animale
- Cambia nome animale
- Mette a dormire/sveglia l'animale
- Usa Item/rimuove vestito
- Acquista Item
- Elimina Item
- Gioca ai minigame

1.4 Requisiti non funzionali

- **Persistenza dei dati:** terminato l'utilizzo del programma, tutti i dati relativi all'utente dovranno essere salvati in modo permanente su un Database relazionale gestito con PostgreSQL.
- **Usabilità:** l'interfaccia grafica dovrà essere semplice e intuitiva tramite comandi espliciti.
- **Affidabilità:** il programma sarà capace di gestire qualsiasi eccezione relativa al database o alla connessione, evitando la perdita di dati dell'utente.
- **Architettura software:** il codice sorgente verrà strutturato seguendo il pattern architetturale BCE + DAO.
- **Vincoli di linguaggio:** Il sistema verrà sviluppato utilizzando il paradigma di programmazione a oggetti, nello specifico usando linguaggio Java, le librerie di Swing UI e i driver JDBC per la connessione al database.
- **Portabilità:** L'applicazione sarà eseguibile su qualsiasi desktop dotato di Java Runtime Environment (JRE) compatibile.

2. DIAGRAMMA DEI CASI D'USO

2.1 Diagramma

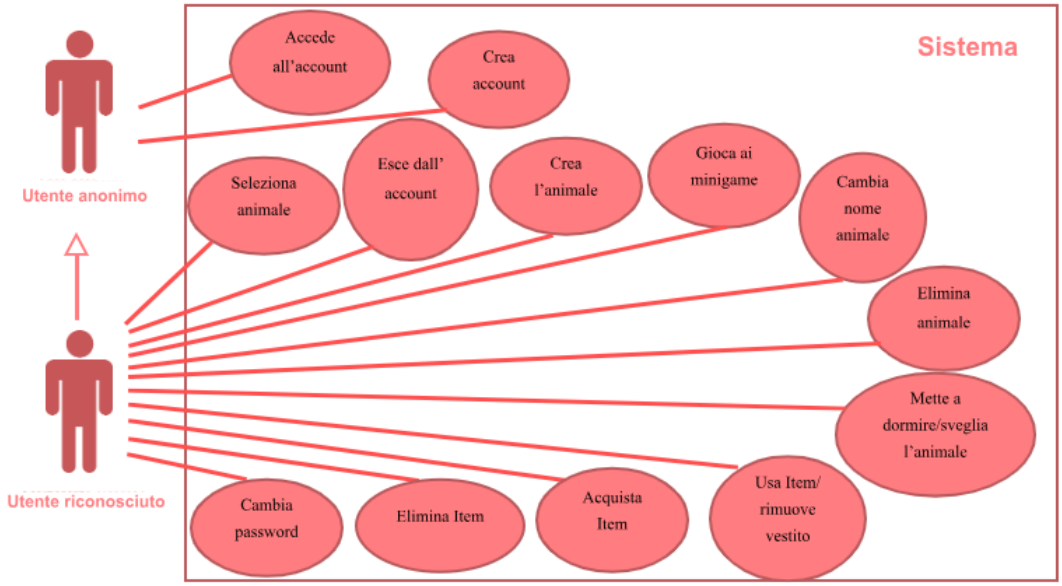


Figura 1: Diagramma dei Casi d'Uso del sistema Tamagotchi.

Note: Il diagramma distingue l'Utente Anonimo (attore non ancora autenticato, che può solo creare un account o effettuare l'accesso) dall'Utente Riconosciuto (attore autenticato). Dopo aver effettuato l'accesso, l'Utente anonimo diventa un Utente riconosciuto capace di svolgere tutte le azioni restanti. Quando l'Utente riconosciuto esce dall'account ritorna ad essere un Utente anonimo.

2.2 Descrizione dei casi d'uso

Crea account:

- **Attori:** Utente anonimo.
- **Precondizioni:** l'utente non deve aver fatto l'accesso per procedere con la creazione dell'account.
- **Scenario Principale:** l'utente inserisce nome utente e password, clicca sul pulsante "Crea Account", viene controllato che il nome utente non sia già stato utilizzato e che le credenziali non abbiano più di 15 caratteri ciascuna. Il sistema salva le credenziali nel database e viene mostrato un messaggio di successo per poi reindirizzare alla schermata di login.
- **Flussi alternativi:** se i campi da riempire sono vuoti, se le credenziali superano il limite di caratteri, o se il nome utente è già utilizzato viene mostrato un messaggio di errore e la registrazione non viene effettuata.

Accede all'account:

- **Attori:** Utente anonimo.
- **Precondizioni:** l'utente deve aver creato un account.
- **Scenario principale:** l'utente inserisce nome utente e password, clicca sul pulsante "Accedi", avviene un controllo se l'account è effettivamente registrato con quelle credenziali nel database, poi viene mostrato un messaggio di successo per poi reindirizzare alla schermata principale.
- **Flussi alternativi:** se i campi da riempire sono vuoti o se le credenziali non vengono riscontrate nel database, viene mostrato un messaggio di errore e l'accesso non viene effettuato.

Crea animale:

- **Attori:** Utente riconosciuto.
- **Precondizioni:** l'utente deve aver effettuato accesso all'account.
- **Scenario principale:** quando l'utente clicca il pulsante "Crea Animale" viene prima verificato nel database che non sia stato superato il numero di animali massimi consentiti, cioè due; poi gli viene mostrata una schermata in cui seleziona fra uno dei due tipi di animali disponibili ("Orso" o "Pinguino") e inserisce il nome dell'animale. Viene verificato che il nome dell'animale sia diverso dai nomi degli altri eventuali animali di quell'utente e che il nome non abbia più di 15 caratteri. Cliccando sul pulsante "Ok", l'animale con le relative informazioni e dati di default viene salvato nel database, associato all'utente.
- **Flussi alternativi:** se il database rileva due animali già presenti, viene mostrato un messaggio di errore e non si procede alla schermata di creazione dell'animale. Se i campi da riempire sono vuoti o il nome supera il limite di caratteri, viene mostrato un messaggio di errore e non viene creato l'animale.

Esce dall'account:

- **Attori:** Utente riconosciuto.
- **Precondizioni:** l'utente deve aver effettuato l'accesso.
- **Scenario principale:** l'utente clicca sul pulsante "Esci", la sua sessione attuale viene interrotta, salvando eventualmente gli ultimi dati e viene reindirizzato alla schermata di accesso.
- **Flussi alternativi:** non presenti.

Seleziona animale:

- **Attori:** Utente riconosciuto.
- **Precondizioni:** l'utente deve aver effettuato l'accesso e creato almeno un animale.
- **Scenario principale:** l'utente clicca su un animale fra quelli posseduti, vengono recuperati i dati relativi all'animale dal database e si viene reindirizzati alla sua pagina di gioco.
- **Flussi alternativi:** non presenti.

Cambia nome animale:

- **Attori:** Utente riconosciuto.
- **Precondizioni:** l'utente deve aver effettuato l'accesso e selezionato un animale.

- **Scenario principale:** l'utente clicca sull'opzione "Cambia nome" nelle impostazioni, inserisce il nuovo nome, clicca sul pulsante "Conferma". Viene verificato che il nome dell'animale sia diverso dai nomi degli altri eventuali animali di quell'utente, e che il nome non sia più lungo di 15 caratteri. Poi viene aggiornato l'attributo nome dell'animale nel database, viene mostrato un messaggio di successo e la GUI si aggiorna di conseguenza.
- **Flussi alternativi:** se i campi da riempire sono vuoti, il nome supera il limite di caratteri o il nuovo nome è identico al nome di un altro animale, viene mostrato un messaggio di errore e il nome non viene cambiato.

Cambia password:

- **Attori:** Utente riconosciuto.
- **Precondizioni:** l'utente deve aver effettuato l'accesso e selezionato un animale.
- **Scenario principale:** l'utente clicca sull'opzione "Cambia password" nelle impostazioni, inserisce la vecchia password e la nuova, clicca sul pulsante "Conferma", se la vecchia password corrisponde all'attuale e la nuova password non ha più di 15 caratteri, viene aggiornato l'attributo password dell'utente nel database, infine viene mostrato un messaggio di successo.
- **Flussi alternativi:** se i campi da riempire sono vuoti, la nuova password supera il limite di caratteri, o la vecchia password non corrisponde alla password attuale, viene mostrato un messaggio di errore e la password non viene cambiata.

Elimina animale:

- **Attori:** Utente riconosciuto.
- **Precondizioni:** l'utente deve aver effettuato l'accesso e selezionato un animale.
- **Scenario principale:** l'utente clicca sull'opzione "Elimina animale" nelle impostazioni, spunta una check box di controllo, clicca sul pulsante "Elimina". Se il check è stato selezionato, allora l'animale viene rimosso dal database, viene mostrato un messaggio di successo e si viene rimandati alla schermata principale con la GUI aggiornata di conseguenza. Se l'animale indossava dei vestiti, questi gli saranno rimossi prima dell'eliminazione.
- **Flussi alternativi:** se il check box non è selezionato viene mostrato un messaggio d'errore e l'eliminazione non viene effettuata.

Mette a dormire/sveglia animale:

- **Attori:** Utente riconosciuto.
- **Precondizioni:** l'utente deve aver effettuato l'accesso e selezionato un animale.
- **Scenario principale:** l'utente clicca sul pulsante "Vai a dormire", l'attributo `dorme` dell'animale viene aggiornato a *true* nel database, il pulsante nella GUI viene aggiornato a "Svegliati" e il livello di energia dell'animale verrà ricaricato progressivamente. Se l'utente clicca sul pulsante "Svegliati", l'attributo `dorme` dell'animale viene aggiornato a *false* nel database, il caricamento di energia viene interrotto, vengono salvati i dati attuali dell'animale e il pulsante nella GUI viene aggiornato a "Vai a dormire".
- **Flussi alternativi:** non presenti.

Acquista Item:

- **Attori:** Utente riconosciuto.
- **Precondizioni:** l'utente deve aver effettuato l'accesso e selezionato un animale.
- **Scenario principale:** l'utente clicca sull'opzione "Negozio", seleziona un'item fra quelli di default e clicca sul pulsante "Compra". Se l'utente ha abbastanza punti, il sistema crea una copia dell'oggetto nell'inventario dell'utente nel database, diminuisce di conseguenza il numero di punti e l'interfaccia degli Item viene aggiornata.
- **Flussi alternativi:** se l'utente non ha selezionato nessuna opzione o il sistema si rende conto che non ha i punti necessari per acquistare l'oggetto, viene mostrato un messaggio di errore e nulla viene acquistato.

Usa Item/rimuove vestito:

- **Attori:** Utente riconosciuto.
- **Precondizioni:** l'utente deve aver effettuato l'accesso e selezionato un animale.
- **Scenario principale:** l'utente clicca sull'opzione "Item", seleziona un'item fra quelli posseduti e clicca sul pulsante "Usa". Il sistema controlla se l'attributo `dorme` dell'animale è *false*. In tal caso, se l'oggetto è un cibo, questo viene eliminato dal database e il valore di fame dell'animale viene aggiornato al nuovo valore; se si tratta di un vestito il sistema controlla che questo non sia indossato da un altro animale e che l'animale non stia indossando già il numero massimo di vestiti consentiti, cioè 2. Superati i controlli, il vestito viene indossato dall'animale attuale, nel database la fame e l'energia massimi vengono aggiornati ai nuovi valori e anche la GUI viene aggiornata. In ogni caso viene mostrato un messaggio di successo. Se l'oggetto selezionato è invece un vestito già indossato dall'animale attuale, il vestito viene rimosso, i valori massimi e o attuali vengono ristabiliti, appare un messaggio di rimozione vestito con successo e anche la GUI viene aggiornata.
- **Flussi alternativi:** se l'utente non ha selezionato nessuna opzione o il sistema si rende conto che l'animale sta dormendo, verrà mostrato un messaggio di errore e l'item non verrà utilizzato. Avverrà lo stesso processo anche nel caso in cui, se l'oggetto selezionato è un vestito, questo è già indossato da un altro animale oppure l'animale attuale sta già indossando più di due vestiti.

Elimina Item:

- **Attori:** Utente riconosciuto.
- **Precondizioni:** l'utente deve aver effettuato l'accesso e selezionato un animale.
- **Scenario principale:** l'utente clicca sull'opzione "Item", seleziona un'item fra quelli posseduti e clicca sul pulsante "Elimina". Il sistema controlla, nel caso in cui si abbia selezionato un vestito, se quest'ultimo non è attualmente indossato da nessun animale: in tal caso l'oggetto viene rimosso dal database, appare un messaggio di successo e la GUI viene aggiornata.
- **Flussi alternativi:** se l'utente non ha selezionato nessuna opzione o il sistema si rende conto che l'eventuale oggetto vestito è indossato, viene mostrato un messaggio di errore e nessun oggetto viene eliminato.

Gioca ai minigame:

- **Attori:** Utente riconosciuto.
- **Precondizioni:** l'utente deve aver effettuato l'accesso e selezionato un animale.
- **Scenario principale:** l'utente clicca sull'opzione "Minigame", seleziona uno dei minigame di default (Sasso Carta Forbici, Lancio della moneta e Slot Machine). Tramite l'interfaccia seleziona le opzioni necessarie, se presenti, e clicca sul pulsante "Gioca". Il sistema controlla se l'animale ha l'energia necessaria per quel minigame e procede con mostrare l'esito casuale del gioco: in caso di vittoria il valore di energia verrà decrementato e i punti aggiornati nel database in base alla ricompensa. Se l'utente perde, verrà solamente decrementata l'energia, mentre se pareggia i valori rimarranno invariati.
- **Flussi alternativi:** se l'utente non ha selezionato nessuna opzione quando necessario, l'animale non ha energia necessaria o l'animale sta dormendo, verrà mostrato un messaggio di errore e l'animale non giocherà.

3. LISTA DELLE FUNZIONI AD ALTO LIVELLO

3.1 Gestione Utente

`recuperaID`:

- **Input:** nome utente (*String*)
- **Output:** idUtente (*int*)
- **Descrizione:** restituisce l'id numerico associato all'Utente nel momento della registrazione. Invia una query al database che recupera esattamente la riga associata a un certo nome utente (essendo univoco si avrà sempre una sola corrispondenza).

`salvaUtente`:

- **Input:** nome utente (*String*), password (*String*)
- **Output:** nessuno
- **Descrizione:** inserisce una nuova riga Utente nel database, dotata di nome utente, password e un id numerico generato automaticamente da PostgreSQL. Questo metodo verrà chiamato durante la creazione di un account.

`controlloLogin`:

- **Input:** nome utente (*String*)
- **Output:** esito (*boolean*)
- **Descrizione:** permette di controllare se nel database è già presente un Utente con lo stesso nome inserito in input. Effettua una query di `SELECT` che cerca un'eventuale riga associata al nome utente, restituendo "true" in caso di riscontro o "false" in caso contrario. Ciò garantisce in primis l'unicità del nome utente.

`aggiornaPassword`:

- **Input:** idUtente (*int*), nuova password (*String*)
- **Output:** nessuno
- **Descrizione:** effettua una query di `UPDATE` della colonna "password" associata alla chiave primaria idUtente.

`cercaUtente`:

- **Input:** nome utente (*String*), password (*String*)
- **Output:** esito (*boolean*)
- **Descrizione:** il metodo cerca la presenza nel database di una riga che abbia nome utente e password corrispondenti a quelle inserite. In caso di riscontro viene restituito "true", in caso contrario "false". Si tratta di un controllo necessario nel momento in cui un Utente tenta l'accesso.

3.2 Gestione Animale

`recuperaId`:

- **Input:** idUtente (*int*), nome animale (*String*)
- **Output:** idAnimale (*int*)
- **Descrizione:** recupera la chiave primaria associata all'animale posseduto dall'utente in input che ha come nome quello inserito (come specificato, gli animali di uno stesso utente hanno nome unico, perciò si avrà sempre solo una corrispondenza).

`salvaAnimale`:

- **Input:** animale (*Animale*, entità del model), idUtente (*int*)
- **Output:** esito (*boolean*)
- **Descrizione:** inserisce un nuovo animale associato all'utente in input: passando l'oggetto, il metodo prima recupera tutte le singole informazioni dell'animale (nome, fame, energia, fameMax, energiaMax, punti, dorme e specie) per poi passarle alla query.

`recuperaListaAnimali`:

- **Input:** idUtente (*int*)
- **Output:** lista animali (*ArrayList*)
- **Descrizione:** effettua una query di `SELECT` per recuperare tutti gli animali associati a un certo utente. Ogni animale trovato verrà creato con le relative informazioni nel model e aggiunto all'ArrayList di ritorno.

`modificaNome`:

- **Input:** idAnimale (*int*), nuovo nome (*String*)
- **Output:** nessuno
- **Descrizione:** effettua una query di `UPDATE` della colonna "nome" associata alla chiave primaria idAnimale.

`eliminaAnimale`:

- **Input:** idAnimale (*int*)
- **Output:** nessuno
- **Descrizione:** effettua una query di `DELETE` della riga associata alla chiave primaria idAnimale.

`aggiornaStatoAnimale`:

- **Input:** animale (*Animale*, entità del model), idAnimale (*int*)
- **Output:** nessuno
- **Descrizione:** recupera tutte le singole informazioni dell'animale in input e poi usa questi dati per aggiornare tutte le colonne relative all'id di quell'animale nel database.

`resetStatoSonno`:

- **Input:** idUtente (*int*)
- **Output:** nessuno
- **Descrizione:** aggiorna la colonna "dorme" di tutti gli animali dell'Utente a *false*. **Nota:** Solitamente il programma in chiusura farà in modo che l'attributo `dorme` dell'animale venga salvato sempre come "false" (per necessità di sistema, all'avvio gli animali dovranno essere tutti svegli). Naturalmente, per errori di connessione al database o arresti anomali dell'applicazione, può capitare che lo stato non venga registrato correttamente. All'avvio successivo della sessione utente, questo metodo viene invocato come *failsafe* preventivo per resettare lo stato di sonno nel database ed allinearlo correttamente alla GUI; questo metodo resetStatoSonno assicura all'avvio l'integrità dei dati.

3.3 Gestione Cibo

`recuperaListaCibo`:

- **Input:** nessuno
- **Output:** lista cibo (*ArrayList*)
- **Descrizione:** effettua una query di `SELECT` per recuperare tutti i cibi memorizzati nel database. Questi verranno istanziati con le relative informazioni nel model e aggiunti all'ArrayList di ritorno. Il metodo permette quindi di ottenere i cibi di default presenti nel negozio.

`recuperaInventarioCibo`:

- **Input:** idUtente (*int*)
- **Output:** inventario cibo (*ArrayList*)
- **Descrizione:** effettua una query di `SELECT` per recuperare tutti i cibi posseduti dall'Utente in input. Questi verranno istanziati con le relative informazioni nel model e aggiunti all'ArrayList di ritorno. Il metodo permette quindi di ottenere i cibi presenti nell'inventario dell'Utente.

`aggiungiAinventarioCibo`:

- **Input:** idUtente (*int*), cibo (*Cibo*, entità del model)
- **Output:** idIstanza (*int*)
- **Descrizione:** effettua una prima query per recuperare l'id del cibo in input tramite il nome (essendo questo valore univoco si avrà un'unica corrispondenza). Segue una seconda query che aggiunge il nuovo cibo al database con l'idUtente, l'idCibo e l'idIstanza generato in automatico. Quest'ultimo sarà anche restituito in output.

`eliminaDaInventario`:

- **Input:** idIstanza (*int*)
- **Output:** nessuno
- **Descrizione:** effettua una query di `DELETE` per rimuovere in modo permanente un cibo consumato/eliminato dall'inventario dell'utente, identificandolo univocamente tramite il suo codice di istanza.

3.4 Gestione Vestito

`recuperaListaVestito`:

- **Input:** nessuno
- **Output:** lista vestiti (*ArrayList*)
- **Descrizione:** effettua una query di `SELECT` per recuperare tutti i vestiti memorizzati nel database. Questi verranno istanziati con le relative informazioni nel model e aggiunti all'ArrayList di ritorno. Il metodo permette quindi di ottenere i vestiti di default presenti nel negozio.

`recuperaInventarioVestito`:

- **Input:** idUtente (*int*)
- **Output:** inventario vestiti (*ArrayList*)
- **Descrizione:** effettua una query di `SELECT` per recuperare tutti i vestiti posseduti dall'Utente in input. Questi verranno istanziati con le relative informazioni nel model e aggiunti all'ArrayList di ritorno. Il metodo permette quindi di ottenere i vestiti presenti nell'inventario dell'Utente.

`aggiungiAinventarioVestito`:

- **Input:** idUtente (*int*), vestito (*Vestito*, entità del model)
- **Output:** idIstanza (*int*)
- **Descrizione:** effettua una prima query per recuperare l'id del vestito in input tramite il nome (essendo questo valore univoco si avrà un'unica corrispondenza). Segue una seconda query che aggiunge il nuovo vestito al database con l'idUtente, l'idVestito e l'idIstanza generato in automatico. Quest'ultimo sarà anche restituito in output.

`eliminaDaInventario`:

- **Input:** idIstanza (*int*)
- **Output:** nessuno
- **Descrizione:** effettua una query di `DELETE` per rimuovere in modo permanente un vestito rimosso/eliminato dall'inventario dell'utente, identificandolo univocamente tramite il suo codice di istanza.

`indossaVestito`:

- **Input:** idIstanza (*int*), idAnimale (*int*)
- **Output:** nessuno
- **Descrizione:** esegue una query di `UPDATE` per associare l'id dell'animale alla riga del vestito nell'inventario, impostando la colonna di controllo (es. `indossato_da`) sul valore dell'idAnimale corrente, attivando così i relativi boost di statistiche.

`rimuoviVestito`:

- **Input:** idIstanza (*int*)
- **Output:** nessuno
- **Descrizione:** esegue una query di `UPDATE` sulla riga del vestito nell'inventario per impostare il valore dell'animale che lo indossa a *NULL*. Questo scollega il vestito dall'animale attuale e ripristina i valori massimi standard di fame ed energia.

3.5 Gestione Minigame

`recuperaMinigame`:

- **Input:** nessuno
- **Output:** lista minigame (*ArrayList*)
- **Descrizione:** effettua una query di `SELECT` sulla tabella dei minigame per estrarre tutti i giochi disponibili di default all'interno del sistema, recuperandone per ciascuno il nome, la ricompensa in punti e il costo energetico richiesto per l'avvio della partita.

4. SCHEMA CONCETTUALE (MODELLO E-R INIZIALE)

Lo schema concettuale è un modello che rappresenta l'organizzazione dei dati del sistema in formato astratto.

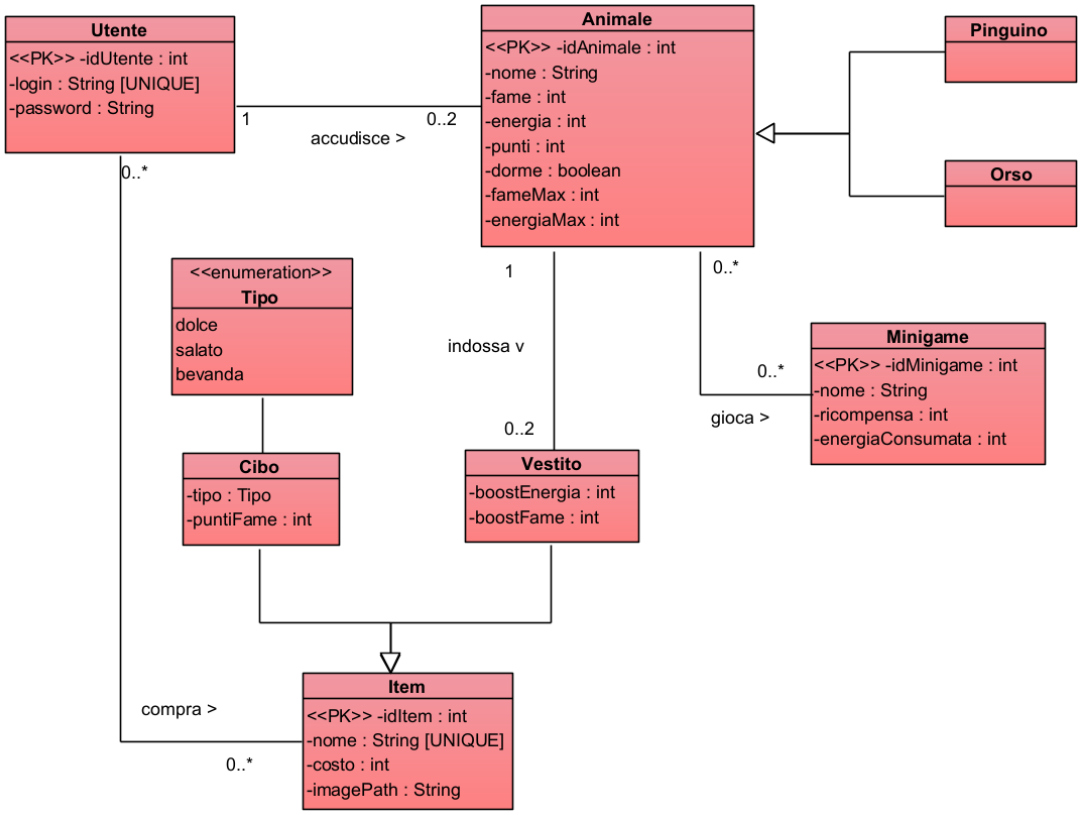


Figura 2: Diagramma delle classi UML iniziale del sistema Tamagotchi.

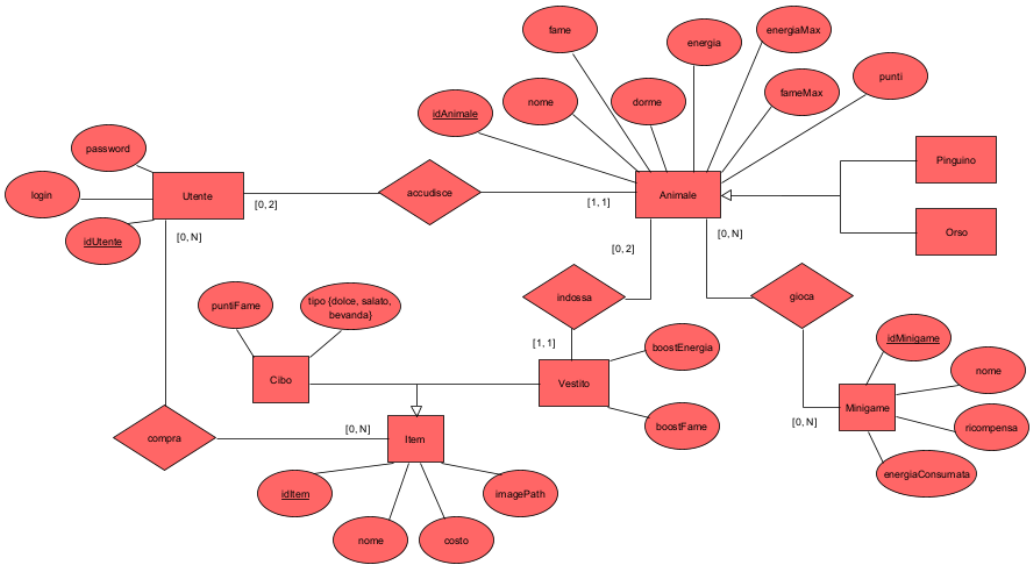


Figura 3: Schema Entità-Relazione (E-R) non ristrutturato.

5. REGOLE DI BUSINESS

5.1 Vincoli di base

- Ogni entità del sistema è identificata con una chiave primaria numerica (**PK**) univoca e non nulla.
- Ogni Animale appartiene a un solo Utente.
- L'attributo `login` dell'entità Utente deve essere univoco, poiché non possono esistere due utenti che hanno lo stesso login.
- La stessa istanza di Vestito non può essere indossata contemporaneamente da più di un animale alla volta.
- Gli attributi `costo` degli Item, `puntiFame` del Cibo, `boostEnergia` e `boostFame` del Vestito, `ricompensa` e `energiaConsumata` del Minigame hanno valore numerico ≥ 0. Invece gli attributi `fameMax` e `energiaMax` dell'Animale hanno valore > 0.

5.2 Vincoli semantici

- Un Utente potrà acquistare un Item solo se l'Animale corrente ha `punti` $\geq$ al costo di quell'Item.
- Un Utente potrà far giocare l'Animale corrente a un Minigame solo se il suo attributo `energia` è $\geq$ all'energiaConsumata di quel Minigame.
- Gli animali di uno stesso utente non devono avere lo stesso nome.
- Gli attributi `login` e `password` di Utente, e l'attributo `nome` dell'Animale, sono stringhe con lunghezza non superiore ai 15 caratteri.
- Gli attributi `energia` e `fame` dell'Animale possono assumere valori compresi fra 0 e, rispettivamente, l'attributo `energiaMax` e `fameMax`.
- Un Utente non può possedere più di 2 animali contemporaneamente.
- Un Animale non può indossare più di 2 istanze di vestiti contemporaneamente.
- Non si può utilizzare un'Item/rimuovere un vestito a un Animale che sta dormendo (`dorme` = *true*).
- Un Animale non può giocare a un Minigame se sta dormendo (`dorme` = *true*).
- Un Vestito indossato non può essere eliminato.

6. SCHEMA CONCETTUALE RISTRUTTURATO

6.1 Diagramma ristrutturato

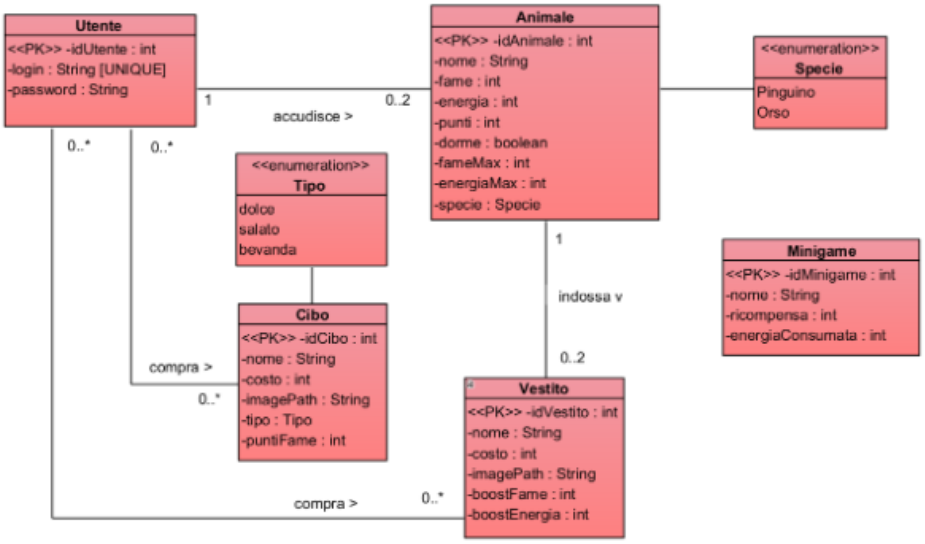


Figura 4: Diagramma ristrutturato del sistema Tamagotchi.

6.2 Ristrutturazione

Note: La generalizzazione di *Pinguino* e *Orso* in *Animale* era necessaria solo a livello di codice Java, per permettere di avere metodi diversi per animali di specie diversa. Non avendo però attributi unici, a livello di gestione dati è inutile mantenere due entità distinte; perciò, sono state collassate entrambe nella classe padre *Animale* in cui la specie viene specificata con una semplice stringa.

Invece la generalizzazione che riguarda le entità *Cibo* e *Vestito* è stata risolta tramite un collasso verso il basso: a causa delle associazioni e dei vincoli specifici riferiti alla classe *Vestito*, è stato deciso di mantenere la distinzione fra le due entità figlie.

L'associazione "gioca" presente nello schema concettuale precedente è stata rimossa, in quanto rappresentava una relazione puramente logica e inutile per la progettazione del database: ai fini del sistema non è necessario tener conto dello storico dei Minigame a cui gioca l'Animale, poiché l'entità *Minigame* ha solo il ruolo di preservare i dati relativi ai giochi di default del sistema.

7. SCHEMA LOGICO DEL DATABASE

Per semplicità visiva dello schema logico viene dato per scontato che tutti gli attributi siano da intendersi `[NOT NULL]` a meno che non venga espressamente specificato il contrario con l'indicazione `[NULL]`. I vincoli di dominio descritti di seguito corrisponderanno a dei vincoli di `CHECK` nativi nel codice SQL.

- **Utente** (idUtente, login `[UNIQUE]`, password)
  - **PK:** idUtente
- **Animale** (idAnimale, nome [UNIQUE], fame, energia, dorme, fameMax, energiaMax, specie, idUtente)
  - **PK:** idAnimale
  - **FK:** idUtente
  - **Vincoli:**
    - fameMax > 0
    - energiaMax > 0
    - $0 \leq$ fame $\leq$ fameMax
    - $0 \leq$ energia $\leq$ energiaMax
    - specie $\in$ {'Pinguino', 'Orso'}
- **Cibo** (idCibo, nome `[UNIQUE]`, costo, imagePath, tipo, puntiFame)
  - **PK:** idCibo
  - **Vincoli:**
    - costo ≥ 0
    - puntiFame ≥ 0
    - tipo $\in$ {'dolce', 'salato', 'bevanda'}
- **Vestito** (idVestito, nome `[UNIQUE]`, costo, imagePath, boostFame, boostEnergia)
  - **PK:** idVestito
  - **Vincoli:**
    - costo ≥ 0
    - boostFame ≥ 0
    - boostEnergia ≥ 0
- **InventarioCibo** (idIstanza, idUtente, idCibo)
  - **PK:** idIstanza
  - **FK:** idUtente riferito a Utente
  - **FK:** idCibo riferito a Cibo
- **InventarioVestito** (idIstanza, idUtente, idVestito, idAnimale `[NULL]`)
  - **PK:** idIstanza
  - **FK:** idUtente riferito a Utente
  - **FK:** idVestito riferito a Vestito
  - **FK:** idAnimale riferito a Animale `[NULL]`
- **Minigame** (idMinigame, nome [UNIQUE], ricompensa, energiaConsumata)
  - **PK:** idMinigame
  - **Vincoli:**
    - ricompensa ≥ 0
    - energiaConsumata ≥ 0

Note: Le entità *Vestito* e *Cibo* servono unicamente a contenere gli oggetti di default disponibili nel negozio. Quando l'Utente acquista un oggetto, potendo possedere più copie di quest'ultimo, verrà creata una nuova istanza di quell'oggetto con un nuovo id per garantirne l'unicità. Quindi sia *InventarioCibo* che *InventarioVestito* generano l'`idIstanza` per ogni nuovo oggetto acquistato.

La tabella *InventarioVestito* gestisce anche l'associazione tra i vestiti acquistati e l'animale che li indossa. Questo è reso possibile dalla presenza della chiave esterna opzionale `idAnimale`: se l'attributo assume valore `NULL`, significa che il vestito si trova nell'inventario dell'utente ma non è attualmente indossato da nessun animale.

8. SPECIFICHE DETTAGLIATE DELLE OPERAZIONI CRITICHE

Vengono analizzate nel dettaglio tre operazioni critiche del sistema, illustrando firma, logica di business, pseudo-codice e operazioni corrispondenti sul database:

8.1 Operazione 1: creaAnimale

Questa operazione permette all'utente loggato di creare un animale, inizializzato con i suoi "stats" di default.

- **Firma:** public void creaAnimale(String tipo, String nome) throws RuntimeException, SQLException
- **Logica di Business:** Il controller si assicura che non ci siano errori che interferiscano con la creazione dell'animale e poi, in base al tipo, definisce l'animale come orso o pinguino. Crea l'oggetto orso/pinguino e lo inserisce nella listaAnimali dell'utente tramite il metodo aggiungiAnimale(), e nel database tramite DAO.
- **OperazioniDB:** Inserimento di record nella tabella Animale tramite AnimaleDAO.salvaAnimale().
- **Pseudocodice:**

```
    -- Nel Controller : creaAnimale ( tipo , nome )
checkAnimali () -- controlla se l'utente possiede gia il
   numero massimo di animali

IF tipo == null OR vuoto THEN THROW ExceptionAnimale ("
   Nessun tipo selezionato. ")
ENDIF
IF nome == null OR vuoto THEN THROW ExceptionAnimale ("
   Nessun nome inserito. ")
ENDIF

IF nome . length () > 15 THEN THROW ExceptionAnimale (" Il
   nome dell'animale deve essere di massimo 15 caratteri. ")
ENDIF

PER OGNI Animale a IN utenteAttuale . getAnimaliPosseduti ()
   DO
    IF a . getNome () == nome THEN
        THROW ExceptionAnimale (" Nome animale gia esistente.
    ")
    ENDIF
ROUTINE

-- crea l'animale in base al tipo selezionato
animale = null

IF " Orso " == tipo THEN
```

```
    animale = NEW Orso ( nome )
ELSE IF " Pinguino " == tipo THEN
    animale = NEW Pinguino ( nome )
ENDIF

-- Operazione DB tramite DAO
animaleDAO . salvaAnimale ( animale , idUtenteAttuale )
aggiungiAnimale ( animale )
```

8.2 Operazione 2: compra

- **Firma:** public void compra(Item item, Animale animale) throws RuntimeException, SQLException
- **Logica di Business:** Il controller verifica se l'item è un cibo o un vestito e gli assegna un idIstanza in base agli altri item posseduti dall'utente, lo aggiunge alla lista degli item dell'utente e ricalcola i punti dell'animale tramite il metodo compraItem(). Salva il risultato di queste operazioni nel database tramite DAO.
- **OperazioniDB:** Inserimento di record nella tabella InventarioCibo o InventarioVestito in base al tipo di item tramite CiboDAO.aggiungiAInventarioCibo() o VestitoDAO.aggiungiAInventarioVestito() rispettivamente.
- **Pseudocodice:**

```
    -- Nel Controller : compra ( item , animale )
IF animale . getPunti () < item . getCosto () THEN
    THROW ExceptionPuntiNonSufficienti (" L'animale non ha
   abbastanza punti! ")
ENDIF

idIstanza = -1

IF item INSTANCEOF Cibo THEN
    -- Operazione DB tramite DAO
    idIstanza = ciboDAO . aggiungiAInventarioCibo (
   idUtenteAttuale , item )
ELSE IF item INSTANCEOF Vestito THEN
    -- Operazione DB tramite DAO
    idIstanza = vestitoDAO . aggiungiAInventarioVestito (
   idUtenteAttuale , item )
ENDIF

utenteAttuale . compraItem ( item , animale , idIstanza )
```

8.3 Operazione 3: usaItem

- **Firma:** public void usaItem(Item item, Animale animale) throws RuntimeException, SQLException
- **Logica di Business:** Il controller verifica se l'item è un cibo o un vestito. Se l'item è un cibo, all'utilizzo ricarica la fame dell'animale e viene rimosso dall'inventario perché consumato. Se l'item è un vestito, all'utilizzo viene indossato dall'animale e vengono applicati i valori di boost. Infine viene aggiornato lo stato dell'animale. Salva il risultato di queste operazioni nel database tramite DAO.
- **OperazioniDB:** Eliminazione di record nella tabella InventarioCibo tramite CiboDAO.eliminaDaInventario() o assegnazione dell' idAnimale al vestito selezionato tramite VestitoDAO.indossaVestito().
- **Pseudocodice:**

```
    --Nel Controller : usaItem ( item , animale )

IF item INSTANCEOF Cibo THEN
    -- Operazione DB tramite DAO
    ciboDAO . eliminaDaInventario ( item . getIdIstanza () )

    -- Logica di business sull'oggetto utente
    utenteAttuale . daiCibo ( item , animale )

ELSE IF item INSTANCEOF Vestito THEN
    -- Controllo e aggiornamento dello stato dell'
   abbigliamento
    IF utenteAttuale . vesti ( item , animale ) THEN
        -- Operazione DB tramite DAO
        vestitoDAO . indossaVestito ( item . getIdIstanza ()
   , idAnimaleAttuale )

        -- Aggiornamento del riferimento dell'item
        item . setIdAnimale ( idAnimaleAttuale )
    ENDIF
ENDIF

-- Salvataggio finale dello stato dell'animale
salvaStatoAnimale ( animale )
```

9. IMPLEMENTAZIONE DEI VINCOLI SUI DATI

9.1 Piano di Implementazione dei Vincoli

Per ciascuna regola di business (vincolo semantico) definita al punto 7, viene presentato il piano dettagliato di implementazione indicando dove avviene la convalida:

Regola	**Collocazione**	**Tecnologia**	**Dettaglio Classe / Metodo**
Energia positiva	Database (Fisico)	`CHECK constraint`	Tabella Animale, `CHECK (energia BETWEEN 0 AND "energiaMax")`.
Fame positiva	Database (Fisico)	`CHECK constraint`	Tabella Animale, `CHECK (fame BETWEEN 0 AND "fameMax")`.
Unicità nome animale	Database (Fisico)	`UNIQUE composto`	Tabella Animale, su `(idUtente, nome)`.
Limite animali	Database (Attivo)	PL/pgSQL Trigger	`trg_limite_due_animali` (BEFORE INSERT).
Limite vestiti indossati	Database (Attivo)	PL/pgSQL Trigger	`trg_limite_due_vestiti` (BEFORE UPDATE).
Blocca elimina vestito	Database (Attivo)	PL/pgSQL Trigger	`trg_no_elimina_vestito_se_indossato` (BEFORE DELETE).
Blocca rimozione vestito	Database (Attivo)	PL/pgSQL Trigger	`trg_no_rimuove_vestito_se_dorme` (BEFORE DELETE).
Effettua acquisto	Applicazione (Java)	Logic Control	Classe `Controller`, metodo `compra()`.
Energia sufficiente	Applicazione (Java)	Logic Control	Classe `Controller`, metodi `giocaSassoCartaForbici()`, `giocaLancioMoneta()`, `giocaSlotMachine()`.
Non usa item se dorme	Applicazione (Java)	Input Validation	Classe `Items`, validazione in Swing.
Non gioca se dorme	Applicazione (Java)	Input Validation	Classe `Tamagotchi`, validazione in Swing.
Lunghezza login e email	Applicazione (Java)	Business Logic	Classe `Utente`, metodo `Utente()`.

Tabella 1: Piano di collocamento dei vincoli nel sistema

9.2 Script dei Trigger in PostgreSQL (PL/pgSQL)

Vengono definiti i codici per la creazione dei trigger e le funzioni associate su PostgreSQL:

```
-- Trigger 1: Controllo limite massimo di animali posseduti da un
    utente
CREATE OR REPLACE FUNCTION controlla_limite_animali()
RETURNS TRIGGER AS $$
DECLARE
    conteggio_animali INT;
BEGIN
    -- conta quanti animali ha gia l'utente che sta subendo l'
   inserimento
    SELECT COUNT(*) INTO conteggio_animali
    FROM "Animale"
    WHERE "idUtente" = NEW."idUtente";

    -- se l'utente ha gia 2 animali, blocca l'inserimento
    IF conteggio_animali >= 2 THEN
        RAISE EXCEPTION 'Errore: L''utente con ID % possiede gia
   il numero massimo consentito di animali (2).', NEW."idUtente";
    END IF;

    -- se il controllo passa, permette il salvataggio della riga
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;


CREATE TRIGGER trg_limite_due_animali
BEFORE INSERT ON "Animale"
FOR EACH ROW
EXECUTE FUNCTION controlla_limite_animali();



-- Trigger 2: Controllo limite massimo di vestiti indossati da un
    animale
CREATE OR REPLACE FUNCTION controlla_limite_vestiti()
RETURNS TRIGGER AS $$
DECLARE
    conteggio_vestiti INT;
BEGIN
    -- conta quanti vestiti ha gia l'animale che sta subendo l'
   inserimento
    SELECT COUNT(*) INTO conteggio_vestiti
    FROM "InventarioVestito"
    WHERE "idAnimale" = NEW."idAnimale";

    -- se l'animale sta gia indossando 2 vestiti, blocca l'
   inserimento
```

```
    IF conteggio_vestiti >= 2 THEN
        RAISE EXCEPTION 'Errore: L''animale con ID % sta gia
   indossando il numero massimo consentito di vestiti (2).', NEW."
   idAnimale";
    END IF;

    -- se il controllo passa, permette il salvataggio della riga
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;


CREATE TRIGGER trg_limite_due_vestiti
BEFORE UPDATE OF "idAnimale" ON "InventarioVestito"
FOR EACH ROW
EXECUTE FUNCTION controlla_limite_vestiti();



-- Trigger 3: Blocca rimozione di un vestito nel caso in cui l
   animale dorma
CREATE OR REPLACE FUNCTION blocca_rimozione_vestito_se_dorme()
RETURNS TRIGGER AS $$
DECLARE
    sta_dormendo BOOLEAN;
BEGIN

    SELECT dorme INTO sta_dormendo
    FROM "Animale"
    WHERE "idAnimale" = OLD."idAnimale";

    -- se l'animale dorme blocca la rimozione
    IF sta_dormendo = TRUE THEN
        RAISE EXCEPTION 'Errore: Non puoi rimuovere un vestito a
   un animale che sta dormendo!';
    END IF;

    -- se non dorme l'operazione procede regolarmente
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_no_rimuove_vestito_se_dorme
BEFORE UPDATE OF "idAnimale" ON "InventarioVestito"
FOR EACH ROW
EXECUTE FUNCTION blocca_rimozione_vestito_se_dorme();
```

```
-- Trigger 4: Blocca eliminazione di un vestito nel caso in cui l
    animale lo stia indossando
CREATE OR REPLACE FUNCTION
   blocca_eliminazione_vestito_se_indossato()
RETURNS TRIGGER AS $$
BEGIN

    -- se l'animale indossa il vestito blocca la cancellazione
    IF OLD."idAnimale" IS NOT NULL THEN
        RAISE EXCEPTION 'Errore: Non puoi eliminare un vestito a
   un animale che lo sta indossando!';
    END IF;

    -- se non lo indossa l'operazione procede regolarmente
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_no_elimina_vestito_se_indossato
BEFORE DELETE ON "InventarioVestito"
FOR EACH ROW
EXECUTE FUNCTION blocca_eliminazione_vestito_se_indossato();
```

10. SCRIPT SQL PER LA CREAZIONE DELLO SCHEMA

Lo script DDL seguente definisce la struttura fisica del database, le tabelle e i relativi vincoli in PostgreSQL:

```
-- Creazione tabella Utente
CREATE TABLE "Utente" (
    login charvar(15) UNIQUE NOT NULL,
    password charvar (15) NOT NULL,
    "idUtente" int GENERATED ALWAYS AS IDENTITY PRIMARY KEY
);

-- Creazione tabella Animale
CREATE TABLE "Animale" (
    "idAnimale" int GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nome charvar(15) UNIQUE ("idUtente", nome) NOT NULL,
    fame int CHECK (fame BETWEEN 0 AND "fameMax") NOT NULL,
    energia int CHECK (energia BETWEEN 0 AND "energiaMax") NOT
   NULL,
    punti int CHECK (punti > 0) NOT NULL,
    dorme boolean DEFAULT false NOT NULL,
    "fameMax" int CHECK ("fameMax" > 0) NOT NULL,
    "energiaMax" int CHECK ("energiaMax" > 0) NOT NULL,
    specie charvar(8) CHECK (specie IN ('Pinguino', 'Orso') NOT
   NULL,
    "idUtente" int REFERENCES "Utente" ("idUtente") ON DELETE
   CASCADE NOT NULL
);

-- Creazione tabella Cibo
CREATE TABLE "Cibo" (
    "idCibo" int GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nome charvar(30) UNIQUE NOT NULL,
    costo int CHECK (costo >= 0) NOT NULL,
    "imagePath" charvar(50) NOT NULL,
    tipo charvar(7) CHECK (tipo IN('dolce', 'salato', 'bevanda')
   NOT NULL,
    "puntiFame" int ("puntiFame" >= 0) NOT NULL,
);

-- Creazione tabella Vestito
CREATE TABLE "Vestito" (
    "idVestito" int GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nome charvar(30) UNIQUE NOT NULL,
    costo int CHECK (costo >= 0) NOT NULL,
    "imagePath" charvar(50) NOT NULL,
    "boostFame" int CHECK ("boostFame" >= 0) NOT NULL,
    "boostEnergia" int CHECK ("boostEnergia" >= 0) NOT NULL,
);
```

```sql
-- Creazione tabella Minigame
CREATE TABLE "Minigame" (
    "idMinigame" int GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nome charvar(50) UNIQUE NOT NULL,
    ricompensa int CHECK (ricompensa >= 0) NOT NULL,
    "energiaConsumata" int CHECK ("energiaConsumata" >= 0) NOT
   NULL
);

-- Creazione tabella InventarioCibo
CREATE TABLE "InventarioCibo" (
    "idIstanza" int GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    "idUtente" int REFERENCES "Utente" ("idUtente") ON DELETE
   CASCADE NOT NULL,
    "idCibo" int REFERENCES "Cibo" ("idCibo") ON DELETE CASCADE
   NOT NULL
);

-- Creazione tabella InventarioVestito
CREATE TABLE "InventarioVestito" (
    "idIstanza" int GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    "idUtente" int REFERENCES "Utente" ("idUtente") ON DELETE
   CASCADE NOT NULL,
    "idVestito" int REFERENCES "Vestito" ("idVestito") ON DELETE
   CASCADE NOT NULL,
    "idAnimale" int REFERENCES "Animale" ("idAnimale") ON DELETE
   SET NULL
);
```

11. SCRIPT DML E POPOLAMENTO DATI

Lo script DML popola il database con record coerenti per cibo, vestiti e minigames:

```
-- Svuotamento preventivo tabelle
DELETE FROM "Utente";
DELETE FROM "Animale";
DELETE FROM "Cibo";
DELETE FROM "Vestito";
DELETE FROM "Minigame";
DELETE FROM "InventarioCibo";
DELETE FROM "InventarioVestito";

-- Inserimento dei cibi di default
INSERT INTO "Cibo" ("idCibo", nome, costo, "imagePath", tipo, "
    puntiFame") VALUES
(1, 'Pizza', 3, 'img/pizza.png', 'salato', 4),
(2, 'Salmone', 5, 'img/salmone.png', 'salato', 8),
(3, 'Broccolo', 2, 'img/broccolo.png', 'salato', 3),
(4, 'Pasta al sugo', 4, 'img/pasta.png', 'salato', 5),
(5, 'Torta', 3, 'img/torta.png', 'dolce', 4),
(6, 'Mela', 2, 'img/mela.png', 'dolce', 3),
(7, 'Baba', 5, 'img/baba.png', 'dolce', 8),
(8, 'Acqua', 1, 'img/acqua.png', 'bevanda', 1),
(9, 'CocaCola', 2, 'img/cocacola.png', 'bevanda', 2),
(10, 'Monster', 2, 'img/monster.png', 'bevanda', 2);

-- Inserimento dei vestiti di default
INSERT INTO "Vestito" ("idVestito", nome, costo, "imagePath", "
    boostFame", "boostEnergia") VALUES
(1, 'Maglia del Napoli', 30, 'img/magliaNapoli.png', 0, 25),
(2, 'Pantaloni skinny', 30, 'img/pantaloniSkinny.png', 20, 10),
(3, 'Scarpe da ginnastica', 30, 'img/scarpeGinnastica.png', 0,
    20),
(4, 'Bavaglino', 20, 'img/bavaglino.png', 15, 0),
(5, 'Cappello da laurea', 40, 'img/cappelloLaurea.png', 25, 25),
(6, 'Corona', 100, 'img/corona.png', 50, 50);

-- Inserimento dei minigames di default
INSERT INTO "Minigame" ("idMinigame", nome, ricompensa, "
    energiaConsumata") VALUES
(1, 'Sasso, Carta, Forbici!', 5, 5),
(2, 'Lancio della moneta', 4, 6),
(3, 'Slot machine', 100, 20);
```